3–4 Years DevOps — Scenario-Based Interview Q&A

1. Developer pushed code, but Jenkins pipeline did not trigger. How will you troubleshoot?
First, I check whether the code was actually pushed to the correct branch and verify the webhook configuration between Git and Jenkins.
Then I check Jenkins job configuration, webhook delivery status, branch filters, credentials, and Jenkins logs.
Git Push
↓
Webhook
↓
Jenkins
↓
Pipeline Trigger
If webhook is working but Jenkins is not triggering, I check whether the Jenkins job is configured for the correct repository and branch.
Simple ga gurthupettuko:
Git Push → Webhook → Jenkins Trigger → Pipeline
Eng: I would first verify the Git push and webhook delivery, then check Jenkins job configuration, branch filters, credentials, and logs to identify why the pipeline was not triggered.

---

2. Jenkins pipeline failed during the Docker build. What will you check?
First, I check the Jenkins console logs to identify the exact Docker build error.
Then I verify:

- Dockerfile syntax
- Base image availability
- Docker build context
- Required files
- Dependencies
- Docker daemon availability
- Registry connectivity if pulling a private image

For example, if the Dockerfile contains:
COPY package.json .
but the file is not available in the build context, the Docker build will fail.
Simple ga gurthupettuko:
Docker Build Fail → **Logs → Dockerfile → Files → Base Image → Dependencies**
Eng: I would analyze the Jenkins console output first and then verify the Dockerfile, build context, base image, dependencies, and Docker environment.

---

3. Jenkins build is successful, but Docker image is not available in the registry. What will you check?
I would first verify whether the image was actually built and tagged correctly.
Then I check the registry login and push command.
Docker Build
↓
Image Tag
↓
Registry Login
↓
Docker Push
I also verify:

- Registry URL
- Repository name
- Image tag
- Credentials
- Network connectivity
- Push permissions

Simple ga gurthupettuko:
Build Success ≠ Push Success
Eng: I would verify the image tag, registry authentication, repository permissions, registry URL, and Jenkins logs to determine why the push failed.

---

4. Kubernetes Pod is in CrashLoopBackOff. How will you troubleshoot?
First, I check the Pod status.
kubectl get pods
Then:
kubectl describe pod <pod-name>
kubectl logs <pod-name>
I check for application startup errors, incorrect environment variables, missing Secrets, configuration problems, dependency failures, wrong commands, or failed probes.
If the container restarted multiple times, I may also check previous container logs.
kubectl logs <pod-name> --previous

Simple ga gurthupettuko:
CrashLoopBackOff → **Describe → Logs → Previous Logs → Root Cause**
Eng: I would inspect Pod events and current or previous container logs, then verify configuration, Secrets, commands, dependencies, and health probes to identify why the container is repeatedly crashing.

---

5. Kubernetes Pod is Running, but application is not accessible. What will you check?
A Running Pod does not necessarily mean the application is reachable.
I check the complete traffic path:
User
↓
Ingress / Load Balancer
↓
Service
↓
Pod
↓
Application
I verify:

- Service exists
- Service selector matches Pod labels
- Endpoints exist
- Service port
- TargetPort
- Container listening port
- Readiness probe
- Ingress rules
- Network policies

Simple ga gurthupettuko:
Running Pod ≠ Working Application
Eng: I would trace the request path from Ingress or Load Balancer to Service and Pod, checking selectors, endpoints, ports, readiness, and application listeners.

---

6. Kubernetes deployment is successful, but users are still getting the old application version. Why?
First, I check whether the new Pods actually contain the expected image version.
kubectl get pods
kubectl describe pod <pod-name>
Then I check:

- Deployment image
- ReplicaSets
- Service selector
- Ingress
- Image tag
- Image pull policy
- CDN/cache if applicable

Sometimes the Deployment is updated but traffic is still reaching old Pods because the rollout has not completed.
Simple ga gurthupettuko:
New Deployment ≠ Automatically means users see New Version
Eng: I would verify the Deployment image, rollout status, ReplicaSets, Service selectors, and any caching layer to determine why traffic is still reaching the old version.

---

7. Terraform plan shows unexpected changes. What will you do?
I would **not immediately run** terraform apply.
First, I inspect the plan and understand why Terraform thinks the resource needs modification.
I check:
Terraform Code
↓
Variables
↓
State
↓
Actual AWS Resource
I verify whether the change came from code modification, variable changes, provider behavior, drift, or state differences.
Simple ga gurthupettuko:
Unexpected Plan → **Understand First → Apply Later**
Eng: I would review the Terraform plan carefully and compare the configuration, state, variables, and actual infrastructure before applying any unexpected change.

---

8. Terraform state is locked. How will you handle it?
First, I determine whether another Terraform operation is currently running.
I don't immediately force-unlock the state because another active operation could still be modifying infrastructure.

If I confirm that the lock is stale and no operation is running, I can use the appropriate Terraform unlock procedure.
Check Active Run
↓
If Running → Wait
↓
If Stale Lock → Unlock Carefully
Simple ga gurthupettuko:
State Lock = **First check another Terraform run**
Eng: I would first verify whether another Terraform operation is active. If the lock is stale and no operation is running, I would safely remove the stale lock using the appropriate Terraform mechanism.

---

9. Ansible playbook is failing on one server but working on others. What will you check?
I compare the failing server with the working servers.
I check:

- SSH connectivity
- Python/interpreter
- OS version
- Package availability
- Permissions
- Variables
- Inventory
- Disk space
- Service status

Working Server
↕ Compare
Failed Server
This helps identify environment-specific differences.
Simple ga gurthupettuko:
One Server Fail → **Working vs Failed Server Compare**
Eng: I would compare the failing host with a successful host, focusing on connectivity, OS, Python, permissions, variables, packages, and service state.

---

10. Production application suddenly becomes slow. How will you troubleshoot?
I would first determine whether the issue is application-level, infrastructure-level, database-level, or network-related.
My troubleshooting flow would be:
User Reports Slowness
↓
Monitoring
↓
CPU / Memory
↓
Application Logs
↓
Database
↓
Network
↓
Recent Deployment
I check metrics such as CPU, memory, latency, request count, error rate, database connections, and application logs.
I also check whether a recent deployment or configuration change caused the problem.
Simple ga gurthupettuko:
Slow App → **Metrics → Logs → DB → Network → Recent Changes**
Eng: I would first identify the scope and timeline of the issue, then correlate infrastructure metrics, application logs, database performance, network behavior, and recent changes to isolate the bottleneck.

---

11. Jenkins pipeline suddenly became very slow. What will you check?
I compare the current pipeline execution time with previous successful builds.
Then I identify which stage became slow.
Pipeline
├── Checkout
├── Build
├── Test ← Slow?
├── SonarQube
├── Docker
└── Deploy
I check Jenkins agent availability, CPU/memory, network connectivity, dependency downloads, Docker builds, test execution, and external service response times.
Simple ga gurthupettuko:
Slow Pipeline → **Find Slow Stage → Check Agent + Dependencies**

Eng: I would compare historical execution times, identify the slow stage, and investigate the Jenkins agent, network, dependency downloads, tests, Docker build, and external integrations.

---

12. Production deployment failed halfway. What will you do?
First, I assess the current application state and user impact.
I identify:

- Which deployment stage failed
- Number of healthy instances/Pods
- Application health
- Error logs
- Previous version availability

If the application is impacted and the previous version is known to be stable, I follow the rollback procedure.
Deployment Failed
↓
Check Impact
↓
Healthy Version?
↙ ↘
Yes No
↓ ↓
Rollback Troubleshoot
Simple ga gurthupettuko:
Failed Deployment → **Check Impact → Stabilize → Rollback/Fix**
Eng: I would first protect availability, assess the deployment state, and if necessary roll back to the last known-good version before investigating and correcting the failed release.

---

13. Docker container works locally but fails in Production. Why?
This commonly happens because the environments are different.
I compare:

- Environment variables
- Secrets
- Configuration
- Image version
- CPU/memory
- Network connectivity
- Mounted volumes
- External dependencies
- Runtime versions

Developer Machine
↓
Works
↓
Production
↓
Fails
↓
Compare Environments
Simple ga gurthupettuko:
Works Local + Fails Prod = **Environment Difference Check**
Eng: I would compare the local and Production environments, especially configuration, secrets, networking, dependencies, resources, and the exact container image.

---

14. How do you handle a failed Production deployment as a 3–4 YOE DevOps engineer?
I would follow a structured incident approach rather than making random changes.
Detect
↓
Assess Impact
↓
Check Logs/Metrics
↓
Identify Recent Change
↓
Rollback if Required → Restore Service → Root Cause Analysis → Permanent Fix

I would communicate the impact and status to the relevant teams while troubleshooting.
After recovery, I would document the root cause and preventive actions.
Simple ga gurthupettuko:
Restore First → RCA Next → Prevention
Eng: My priority during a Production deployment failure is to minimize user impact, restore service safely, identify the root cause, and implement preventive actions to avoid recurrence.